

Group Project Proposal

Improving Repository Navigation and Bug Localization in Minimal Software Engineering Agents

Group ID: 25

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

Administrative Information

Group members	ZHANG YUHANG(A0351628B) WANG CHI(A0357310M) CHEN WENWEN(A0351705J) JIANG ZHANGZHANG(A0355183B) CHEN XINGYU(A0355370E)
Project direction	Software Engineering Agents – Repository Navigation and Bug Localization.
Repository	https://github.com/235559649/DSA4213_group_work

Project Summary

Software engineering agents such as mini-SWE-agent can autonomously explore repositories, modify code, and execute tests, but repository exploration is largely driven by free-form shell interaction. This project studies whether structured repository navigation and failure-aware bug localization can improve the efficiency and reliability of software repair agents. We will extend mini-SWE-agent with a Python-oriented repository indexer that ranks relevant files and functions based on issue descriptions, code structure, and runtime failure evidence. We will compare the proposed agent with the original mini-SWE-agent on real bug-fixing tasks, evaluating repair success rate, localization accuracy, number of explored files, shell commands, token usage, runtime, and failure patterns.

1. Problem, Research Question, and Motivation

TODO: CHEN WENWEN

Software engineering agents must often explore a large repository before they can determine which files or functions are responsible for a reported bug. A minimal agent such as mini-SWE-agent can perform this exploration using shell commands and language-model reasoning, but the navigation process is not explicitly optimized for locating relevant code.

Our project focuses on the repository exploration and bug localization stage of the software repair process. Rather than replacing the existing repair capability of mini-SWE-agent, we investigate whether additional repository structure and runtime failure evidence can help the agent reach relevant code more efficiently.

TODO: Add a short motivating example showing an issue where the agent must search through several files before reaching the correct function.

1.1 Primary Research Question

RQ1. Can structured repository navigation and runtime failure evidence help mini-SWE-agent localize relevant code more efficiently without reducing bug-fixing success?

1.2 Secondary Research Questions

RQ2. Does improved localization reduce repository exploration cost, measured by the number of files inspected, shell commands executed, token usage, and runtime?

RQ3. Which information sources contribute most to successful localization: textual relevance to the issue, static repository structure, or runtime failure evidence?

1.3 Definition of Success

The project will be considered successful if the proposed system can either improve bug localization and repair performance, or achieve comparable repair performance while reducing unnecessary repository exploration.

TODO: Define quantitative success criteria after pilot experiments, for example an improvement in Top- k localization accuracy or a reduction in exploration steps.

2. Related Work and Proposed Contribution

2.1 mini-SWE-agent

Software engineering agents alternate between model decisions, repository operations, and execution feedback. SWE-agent shows that the agent–computer interface (ACI) affects repository-level software repair through the design of commands and feedback^[1]. Building on this insight, we adopt **mini-SWE-agent** as our baseline: a separate, minimal implementation combining a language model, a command-execution environment, and a linear interaction history^[2]. Its shell interface supports the core operations needed for repair—repository search, code inspection, editing, and test execution. By default, the v2 implementation issues these operations through native bash tool calls, while legacy configurations instead parse commands from model-generated text^[3].

The teal path in Figure 1 shows the agent loop: *decide, act, observe, and update history*. Given the issue and interaction history, the model selects an action, which the environment executes as a shell command. The resulting output, errors, and test results are formatted as observations and appended to the history, alongside the model’s response, to inform the next decision^[2]. Across iterations, the agent can locate relevant code, investigate or reproduce a bug, edit files, and run accessible tests. These are available actions rather than a fixed sequence: new evidence may prompt further inspection or another edit. The loop continues until submission or another stopping condition, such as a resource limit. The repository diff provides a candidate patch whose correctness is assessed separately.

Our extension preserves the model adapter, execution environment, and repair loop. It supplies ranked localization context before model decisions and updates that context when execution yields usable failure evidence. The baseline retains ordinary execution feedback, including failures, so the comparison isolates the added value of explicit candidate reranking.

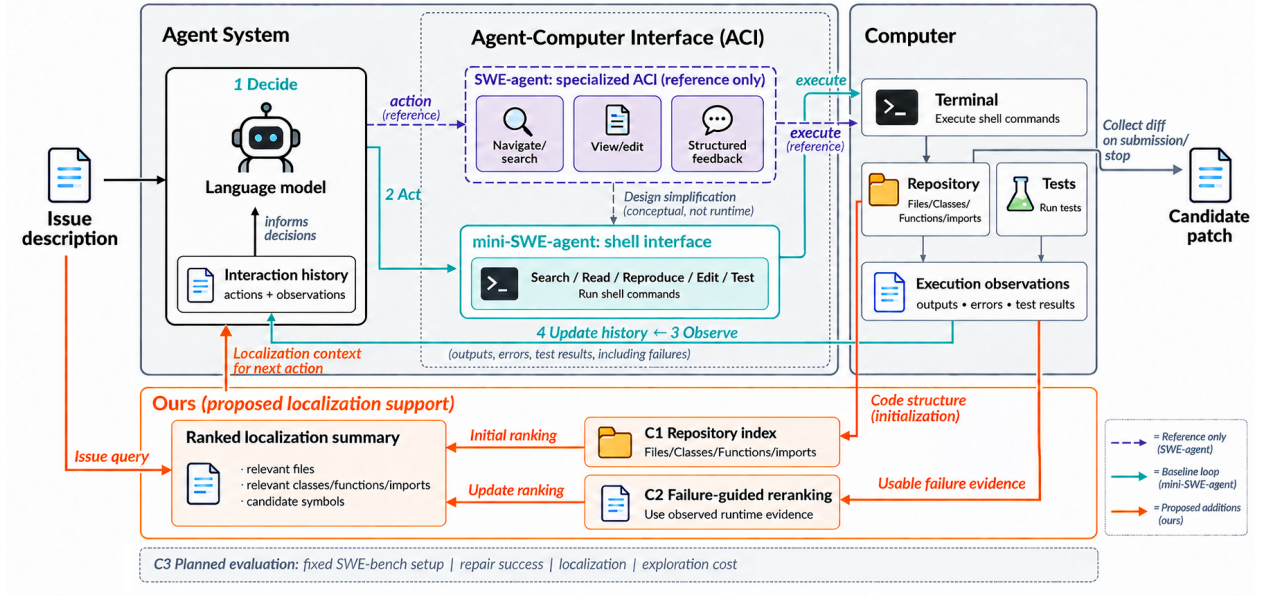


Figure 1: Overview of mini-SWE-agent and the proposed localization extension.

2.2 SWE-bench

SWE-bench evaluates repository-level software repair using real GitHub issues and executable tests^[4]. We propose to use **SWE-bench Verified**, a human-screened subset of 500 tasks from Python repositories^[5]. Its screening helps reduce ambiguity in issue descriptions and problems with evaluation tests. Its Python scope also matches our repository indexer, making it suitable for examining whether improved localization translates into successful repair.

From this task pool, we initially plan to select approximately 20 tasks for development and 100 additional tasks for evaluation, covering multiple repositories. The development tasks will support debugging and refinement of the localization components. The evaluation tasks will be reserved for comparing the baseline and proposed variants after the method is fixed. We may adjust these target sizes based on runtime and API costs measured during development, before fixing the evaluation task list. The two sets will remain disjoint, and all compared systems will use the same evaluation tasks.

2.3 Repository Localization Methods

Repository localization uses issue descriptions, code structure, and execution evidence to identify relevant files and functions. These information sources support complementary stages of the search: textual relevance provides initial candidates, structural relationships guide further inspection, and runtime observations help revise the search as repair proceeds, as shown in Table 1.

Table 1: Prior methods relevant to this study.

Category	Approach	Relevant mechanism	Role in this study
Agent framework	mini-SWE-agent ^[2]	Shell actions and execution feedback in a minimal agent loop.	Provides the baseline repair loop.
Textual retrieval	BugLocator ^[6]	File ranking using textual similarity and historical bug information.	Motivates the lexical localization comparison.
Staged localization	Agentless ^[7]	Hierarchical localization followed by repair and validation.	Separates candidate selection from patch correctness.
Structural navigation and reasoning	LocAgent ^[8]	Navigation over code entities and dependency relationships.	Motivates structural support for code search.
	CoSIL ^[9]	Iterative code-graph search with context pruning.	Motivates refinement of localization context.
	RepoNav ^[10]	File-centered organization of snippets and candidate functions.	Motivates structured presentation of candidates.
	GraphLocator ^[11]	Graph-guided reasoning about symptoms and edit locations.	Distinguishes observed symptoms from repair targets.
Execution-informed localization	AutoCodeRover ^[12]	Structural search with optional spectrum-based fault localization.	Connects code structure with test evidence.
	IssueExec ^[13]	Test retrieval and execution-trace analysis.	Links execution evidence to candidate edit locations.
Proposed integration	This study (planned)	Repository indexing and reranking from observed failures.	Updates localization context within the mini-SWE-agent loop.

Textual retrieval and staged localization. BugLocator ranks source files using report-code textual similarity and historical bug information^[6]. Agentless organizes issue resolution into localization, repair, and validation, progressively narrowing the code considered for modification^[7]. Together, these approaches provide a basis for selecting candidate locations before generating a patch. We therefore include a lexical localization condition to examine how much guidance issue-code matching provides within mini-SWE-agent. Assessing localization alongside repair will show whether improved candidate selection translates into successful fixes.

Structural navigation and evidence organization. LocAgent represents code entities and dependencies in a heterogeneous graph, allowing the agent to navigate relationships between relevant locations^[8]. CoSIL combines iterative code-graph search with context pruning to refine the information considered during localization^[9]. RepoNav organizes retrieved snippets around files and candidate functions, supporting comparison between possible targets within a file^[10]. These studies show how code relationships and candidate organization can guide repository exploration beyond textual matching. They motivate our Python repository index, which records files, classes, functions, and imports, and supplies a ranked localization summary for subsequent agent decisions.

Execution-informed localization. AutoCodeRover combines structure-aware code search with optional spectrum-based fault localization^[12]. IssueExec retrieves relevant tests and analyzes

their execution traces to identify candidate edit locations^[13]. These methods establish execution as a source of evidence linking an issue to the code involved in its behavior. GraphLocator further motivates examining the relationship between observed symptoms and locations requiring modification^[11]. Building on this rationale, our proposed component maps tracebacks, failing-test identifiers, and error messages observed during repair to indexed code entities. It then updates the candidate rankings and supplies the revised summary to the agent before its next decision.

Positioning of this study. The proposed system combines initial localization from repository structure with ranking updates from the agent’s own execution trajectory. Its focus is the connection between observed failures, indexed code entities, and the localization context used in subsequent decisions. The original mini-SWE-agent provides the repair loop and ordinary execution feedback; our extension adds an explicit ranking update mechanism. This design supports a controlled study of how structured localization context affects repair success and exploration cost. Table 1 summarizes the relevant mechanisms and the design choices they motivate.

2.4 Our Contribution

Prior work already supports structural navigation and execution-informed localization. Our contribution is a focused extension and controlled study within mini-SWE-agent, asking whether these signals improve localization efficiency without reducing repair success. We retain the three contributions defined in the group proposal:

- **C1: Repository indexing.** A Python-oriented index of files, classes, and functions that supports localization from the issue description.
- **C2: Failure-aware refinement.** A mechanism that maps observed runtime failures to code entities and updates suspicious-location rankings.
- **C3: Empirical evaluation.** A comparison with the original mini-SWE-agent of repair performance and repository exploration efficiency.

3. Agent and Task Setup

TODO: ZHANG YUHANG

3.1 Task

The agent receives a software issue together with a Python repository. Its goal is to identify the code relevant to the issue, produce a patch, and validate the patch through execution or tests.

Input: a natural-language issue description and the corresponding repository.

Output: a proposed source-code patch and an execution trajectory containing repository exploration, edits, and test results.

3.2 Operating Environment

The repair agent will operate inside an isolated execution environment with access to the repository, shell commands, Python execution, Git, and the repository’s available test framework.

TODO: Specify the exact environment, e.g., Docker / SWE-bench containers, Python versions, and available system tools.

3.3 Agent and Model

The original mini-SWE-agent will be used as the software repair baseline. Our system will augment its repository exploration process with structured localization information.

TODO: Specify the LLM(s) used in experiments.

TODO: Specify whether all compared systems will use the same model, temperature, token budget, and prompts to ensure a controlled comparison.

3.4 Agent Tools

The proposed system may provide the agent with the following sources of information:

- shell-based repository exploration;
- file, class, and function indexing;
- Python AST and import information;
- issue-to-code relevance scores;
- test execution results; and
- traceback and runtime failure evidence.

TODO: Remove any tool that is not implemented in the final system and describe the exact interface exposed to the agent.

3.5 Benchmark and Task Selection

The initial study will focus on Python repositories so that Python ASTs, tracebacks, and testing tools can be exploited consistently.

TODO: Define the exact benchmark subset and task selection criteria.

TODO: Specify the number of tasks used for pilot development and final evaluation.

3.6 Stopping Conditions and Failure Recovery

An agent run may terminate when the target tests pass, when the agent explicitly submits a final patch, or when a predefined resource budget is exhausted.

When execution fails, the system can extract information from the failure and use it to update the localization ranking before the next repair attempt.

TODO: Define maximum agent steps, token budget, runtime limit, and the exact condition used to determine successful repair.

4. Proposed System or Study Design

TODO: CHEN XINGYU

The proposed workflow consists of an initial repository analysis stage, a localization stage, the existing mini-SWE-agent repair process, and an optional refinement stage based on runtime failures.

TODO: Insert system architecture diagram: Issue → Repository Index → Initial Localization → mini-SWE-agent → Test Execution → Failure-Aware Refinement → Repair.

Figure 2: Proposed system architecture.

4.1 Baseline

The baseline is the original mini-SWE-agent operating with its standard repository exploration and repair loop.

TODO: Describe the exact baseline configuration and ensure it uses the same LLM and execution budget as the proposed system.

4.2 Repository Indexing

Before repair begins, the repository indexer analyzes the Python repository and records structural information such as:

- file paths;
- classes;
- functions and methods;
- import relationships; and
- selected code metadata relevant to localization.

TODO: Define the actual internal representation. Decide whether this will be a table, graph, JSON index, or another structure.

TODO: Decide whether call relationships or inheritance information are included. Avoid adding them unless they are actually used.

4.3 Initial Localization

Given an issue description, the localization module ranks repository files and functions according to their estimated relevance to the issue.

The result will be a ranked list of suspicious locations that can be provided to the repair agent as additional context.

TODO: Define the localization scoring function.

TODO: Decide whether the initial implementation includes keyword search, semantic embeddings, structural signals, or a hybrid method.

4.4 Failure-Aware Refinement

If a reproduction attempt or test execution fails, the system extracts runtime evidence such as:

- exception type;
- traceback frames;
- files and functions appearing in the traceback;
- failing test names; and
- relevant error messages.

This information is then used to update the ranking of suspicious code locations.

TODO: Define how runtime evidence modifies the original ranking.

TODO: Provide one concrete example trajectory in the final proposal if space permits.

4.5 Repair Loop

The localized context is passed to mini-SWE-agent, which investigates the repository, modifies source code, and executes tests. If the patch fails, failure information may be returned to the localization component and a new repair attempt can be made.

TODO: Specify whether localization is performed only once or repeatedly during the trajectory. This decision is important for the final system design.

4.6 Optional Extension: Reviewer Agent

If time permits, an independent reviewer agent may inspect a proposed patch before final submission.

The reviewer would focus on correctness, whether the reported failure has actually been addressed, potential regressions, and relevant edge cases.

TODO: Keep this as an optional extension. Do not make it part of the main claim unless the core localization system is completed early.

4.7 Comparisons and Ablations

The planned controlled conditions are:

- **Baseline A:** original mini-SWE-agent;
- **Baseline B:** mini-SWE-agent with simple lexical / keyword localization;
- **Method C:** mini-SWE-agent with repository-structure-aware localization;

- **Method D:** mini-SWE-agent with repository-structure-aware localization and failure-aware refinement.

Possible ablations will remove individual information sources from the full system.

TODO: Finalize comparison conditions after implementing a pilot version. Do not promise more baselines than the group can realistically run.

TODO: Possible ablations: no structural information, no failure evidence, or file-level rather than function-level localization.

5. Preliminary Evaluation Plan

TODO: JIANG ZHANGZHANG

The evaluation is designed to distinguish whether the proposed method improves actual bug fixing, localization quality, or only repository exploration efficiency.

5.1 Repair Performance

Potential measures include:

- issue resolution rate;
- target tests passed;
- regression-test performance; and
- number of successful repairs within the resource budget.

TODO: Define the official repair-success criterion using the selected benchmark’s evaluation harness.

5.2 Localization Performance

Potential localization metrics include:

- Top-1 relevant-file accuracy;
- Top- k relevant-file accuracy;
- function-level localization accuracy; and
- rank of the ground-truth modified location.

TODO: Define ground-truth localization from benchmark patches and decide the value(s) of k .

5.3 Exploration Efficiency

We will measure the amount of repository exploration required before the agent reaches relevant code.

Potential measures include:

- number of files opened;
- number of shell commands executed;
- number of agent / LLM steps;
- token usage;
- runtime; and
- estimated API cost.

TODO: Select the final subset of efficiency measures to avoid reporting too many redundant metrics.

5.4 Agent-Trajectory Analysis

Aggregate metrics will be complemented with representative successful and failed trajectories. We will examine, for example, whether failed runs repeatedly inspect irrelevant files, fail to use runtime evidence, or reach the correct code but produce an incorrect patch.

TODO: Define a small failure taxonomy after observing pilot trajectories.

5.5 Variability and Experimental Protocol

Because LLM-based agents may produce different trajectories across runs, the comparison should control model configuration and resource budgets.

TODO: Decide the number of repeated runs per task, subject to API and compute cost.

TODO: Specify model version, temperature, maximum steps, token budget, and random seeds where applicable.

6. Risks, Limitations, and Responsible Practice

TODO: CHEN WENWEN

- **LLM nondeterminism.** Repeated runs may produce different trajectories and repair results. We will use controlled model settings and repeated evaluation where resources permit.
- **Benchmark contamination.** Public repositories or issue solutions may have appeared in model training data, so benchmark performance should not be interpreted purely as unseen-code generalization.
- **Execution safety.** Software engineering agents can execute shell commands. Experiments will therefore be conducted in isolated environments with restricted permissions.

- **API and compute cost.** Full agent trajectories can require substantial token usage and execution time. A fixed experimental budget will be used.
- **Environment failures.** Some benchmark repositories may fail because of dependency or setup problems unrelated to the agent itself. These cases will be logged separately where possible.
- **Python-specific scope.** The initial system focuses on Python repositories, so conclusions may not directly generalize to other programming languages.

6.1 Safeguards

Experiments will use isolated execution environments, restricted credentials and permissions, fixed resource budgets, and public benchmark repositories only. No private repositories, credentials, or confidential source code will be included.

TODO: Confirm the exact sandbox/container mechanism used by the final implementation.

References

- [1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [2] mini-SWE-agent contributors. mini-SWE-agent. Software repository, n.d.. URL <https://github.com/SWE-agent/mini-swe-agent>. Accessed 2026-09-22.
- [3] mini-SWE-agent contributors. mini-SWE-agent v2.0 Migration Guide. Official documentation, n.d.. URL https://mini-swe-agent.com/latest/advanced/v2_migration/. Accessed 2026-09-22.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [5] OpenAI. Introducing SWE-bench Verified. Official benchmark announcement, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed 2026-09-22.
- [6] Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering*, pages 14–24, 2012. doi: 10.1109/ICSE.2012.6227210. URL https://ink.library.smu.edu.sg/sis_research/1531/.
- [7] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents, 2024. URL <https://arxiv.org/abs/2407.01489>.
- [8] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-Guided LLM Agents for Code Localization, 2025. URL <https://arxiv.org/abs/2503.09089>.

- [9] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue Localization via LLM-Driven Iterative Code Graph Searching, 2025. URL <https://arxiv.org/abs/2503.22424v2>.
- [10] Hongzheng Chai, Jiakun Li, Hongyue Yu, and Yuan Yuan. RepoNav: From Snippet Retrieval to File-Centered Repository Navigation for Code Agents, 2026. URL <https://arxiv.org/abs/2609.08355>.
- [11] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. GraphLocator: Graph-guided Causal Reasoning for Issue Localization, 2025. URL <https://arxiv.org/abs/2512.22469>.
- [12] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement, 2024. URL <https://arxiv.org/abs/2404.05427>.
- [13] Jiawei Liu, Yun Lin, Chenyan Liu, Yu Qian, Yiming Liu, Jiaxin Chang, Weinan Zhang, and Linpeng Huang. IssueExec: A Test-Driven Approach for Localizing Software Engineering Issues, 2026. URL <https://arxiv.org/abs/2607.17286>.